



Modelos de Regressão

ENG1470

Prova Prática 01

Aluno(a)	Paloma Fernanda Loureiro Sette - 1621595
Professor(a)	Cristiano Fernandes

Rio de Janeiro, 28 de Abril de 2023

Sumário

Lista de Figuras	1
1 - Questão 01	2
2 - Questão 02	4
3 - Questão 03	5
4 - Questão 04	8
4.1 - Item (i)	8
4.2 - Item (ii)	9
5 - Questão 05	10
5.1 - Teste de não-linearidade	10
5.2 - Teste de Interação	11
6 - Questão 06	13
7 - Questão 07	14
8 - Questão 08	18
9 - Questão 09	19
10 - Questão 10	22
11 - Questão 11	23

Lista de Figuras

1	Resultado da Regressão Linear Múltipla	9
2	Resultado - Questão 07	17
3	Resultado - Questão 08	19
4	Resultado de y_{pred} com os novos dados - questão 11	26
5	Gráfico Q-Q plot	27

Introdução

Este relatório refere-se à primeira prova prática da disciplina de Modelos de Regressão (ENG1470) de 2023.1.

Para tal, utilizarei uma base de dados cujo dataset encontra-se neste [link](#).

O conjunto de dados "Beer Consumption - São Paulo" se refere ao consumo de cerveja na cidade de São Paulo, Brasil. Ele contém informações diárias sobre o consumo de cerveja, bem como dados sobre as condições climáticas, como temperatura média, temperatura mínima, temperatura máxima e precipitação de chuva. Além disso, há uma variável indicadora binária que indica se o dia é um fim de semana ou não.

O objetivo do conjunto de dados é explorar a relação entre o consumo de cerveja e as condições climáticas, a fim de entender como fatores como temperatura e precipitação podem afetar o consumo de cerveja. Esse tipo de análise pode ser útil para empresas de cerveja e bares, por exemplo, para prever a demanda por cerveja e ajustar suas estratégias de negócios de acordo.

Para a resolução deste teste, utilizarei a linguagem de programação Python.

1 - Questão 01

Inicialmente obtenha para todas as variáveis da sua base de dados (a variável dependente e as independentes/regressores) a matriz de correlação.

Para calcular a matriz de correlação para todas as variáveis do conjunto de dados "Beer Consumption - São Paulo", podemos utilizar a linguagem Python e a biblioteca pandas.

Utilizei os seguintes comandos:

```

import pandas as pd
import numpy as np

# carregando o conjunto de dados
df = pd.read_excel("Consumo_cerveja.xlsx")
df['data'] = pd.to_datetime(df['data'], format='%Y-%m-%d')

#fazendo o tratamento necessario nas colunas
for i, temp_med in enumerate(df['temp_med']):
    if isinstance(temp_med, str):
        df.loc[i,
                'temp_med'] = float(temp_med.replace(',','.', ''))
df['temp_med'] = df['temp_med'].astype(float)
for i, temp_min in enumerate(df['temp_min']):
    if isinstance(temp_min, str):
        df.loc[i,
                'temp_min'] = float(temp_min.replace(',','.', ''))
df['temp_min'] = df['temp_min'].astype(float)
for i, temp_max in enumerate(df['temp_max']):
    if isinstance(temp_max, str):
        df.loc[i,
                'temp_max'] = float(temp_max.replace(',','.', ''))
df['temp_max'] = df['temp_max'].astype(float)
for i, chuva_mm in enumerate(df['chuva_mm']):
    if isinstance(chuva_mm, str):
        df.loc[i,
                'chuva_mm'] = float(chuva_mm.replace(',','.', ''))
df['chuva_mm'] = df['chuva_mm'].astype(float)
for i, consumo in enumerate(df['consumo']):
    if isinstance(consumo, str):
        df.loc[i,
                'consumo'] = float(consumo.replace(',','.', ''))
df['consumo'] = df['consumo'].astype(float)

# selecionando as colunas numericas
df_num = df.select_dtypes(include=[np.number])

# criando a matriz de correlacao
corr_matrix = df_num.corr()

# imprimindo a matriz de correlacao
print(corr_matrix)

```

Portanto, temos a matriz de correlação abaixo:

	temp_med	temp_min	temp_max	chuva_mm	fds	consumo
temp_med	1.000000	0.862752	0.922513	0.024416	-0.050803	0.574615
temp_min	0.862752	1.000000	0.672929	0.098625	-0.059534	0.392509
temp_max	0.922513	0.672929	1.000000	-0.049305	-0.040258	0.642672
chuva_mm	0.024416	0.098625	-0.049305	1.000000	0.001587	-0.193784
fds	-0.050803	-0.059534	-0.040258	0.001587	1.000000	0.505981
consumo	0.574615	0.392509	0.642672	-0.193784	0.505981	1.000000

2 - Questão 02

Escolha as 5 variáveis independentes com maior coeficiente de correlação simples com y (algumas podem ser dicotômicas). Se a correlação de uma determinada dummy de um regressor com y tiver magnitude razoável e for escolhida, todas as outras dummies associadas a esse regressor qualitativo devem ser incluídas. Para a variável y e os regressores escolhidos especifique as unidades de medida. Em seguida especifique a FRP linear múltipla, com as expectativas dos sinais dos coeficientes de regressão de cada variável, justificando.

Peço desculpas, a minha resposta anterior estava incorreta. Para escolher as 5 variáveis independentes com maior coeficiente de correlação com a variável dependente, devemos considerar as variáveis com maiores valores absolutos de correlação.

Nesse caso, as 5 variáveis independentes com maiores valores absolutos de correlação com a variável dependente (consumo) são:

1. temp_max (correlação positiva forte)
2. temp_med (correlação positiva moderada)
3. fds (correlação positiva moderada)
4. temp_min (correlação positiva fraca)
5. chuva_mm (correlação negativa fraca)

As unidades de medida são:

1. Temperatura máxima (temp_max) - unidade: graus Celsius
2. Temperatura média (temp_med) - unidade: graus Celsius
3. Fim de semana (fds) - variável dicotômica (1 se é fim de semana, 0 caso contrário)
4. Temperatura mínima (temp_min) - unidade: graus Celsius
5. Precipitação (chuva_mm) - unidade: milímetros.
6. Consumo (consumo) - unidade: litros.

Portanto, a FRP (Função de Regressão Paramétrica) linear múltipla seria:

$$\text{Consumo} = \beta_0 + \beta_1 * \text{temp_med} + \beta_2 * \text{temp_min} + \beta_3 * \text{temp_max} + \beta_4 * \text{chuva_mm} + \beta_5 * \text{fds} + \varepsilon \quad (1)$$

Sendo β_0 o intercepto, β_1 o coeficiente de regressão da temperatura média, β_2 o coeficiente de regressão da temperatura mínima, β_3 o coeficiente de regressão da temperatura máxima, β_4 o coeficiente de regressão da quantidade de chuva, β_5 o coeficiente de regressão do indicador de final de semana e ε o erro aleatório.

As expectativas dos sinais dos coeficientes de regressão seriam:

- β_1 (temperatura média): positivo, já que é esperado que o consumo de cerveja aumente com o aumento da temperatura média.
- β_2 (temperatura mínima): positivo, já que é esperado que o consumo de cerveja aumente com o aumento da temperatura mínima.
- β_3 (temperatura máxima): positivo, já que é esperado que o consumo de cerveja aumente com o aumento da temperatura máxima.
- β_4 (quantidade de chuva): negativo, já que é esperado que o consumo de cerveja diminua com o aumento da quantidade de chuva.
- β_5 (indicador de final de semana): positivo, já que é esperado que o consumo de cerveja seja maior nos finais de semana.

3 - Questão 03

Estime uma regressão linear simples para cada um dos regressores individualmente. Observe: a significância do coeficiente de cada regressor no nível 5% via teste t (teste bilateral), o sinal da estimativa de cada coeficiente e o R² do modelo. Apresente os resultados em forma de tabela. Se vc fosse escolher um modelo com um único regressor, qual vc escolheria? Justifique.

Para este item, importei a biblioteca *statsmodels* no Python:

```
import statsmodels.api as sm
```

Após isso, utilizei o seguinte trecho de código, onde eu crio uma função que recebe como parâmetro uma variável que será relativa a cada um dos nossos regressores, extraindo uma tabela para cada um:

```

def create_simple_lin_reg(variable):
    # criando a tabela com os resultados
    table_data = []
    var = variable

    # estima o modelo de regressao linear simples
    X = df[var]
    y = df['consumo']
    X = sm.add_constant(X)
    model = sm.OLS(y, X).fit()

    # obtem os resultados
    coeficiente = model.params[var]
    erro_padrao = model.bse[var]
    t_value = model.tvalues[var]
    p_value = model.pvalues[var]
    r_squared = model.rsquared

    # adiciona os resultados a tabela
    row = [var, coeficiente,
           erro_padrao, t_value,
           p_value, r_squared]
    table_data.append(row)

    table = pd.DataFrame(table_data, columns=['Regressor',
                                              'Coeficiente',
                                              'Erro_Padrao',
                                              't',
                                              'Valor-p',
                                              'R2'])

    table = table.set_index('Regressor')
    return table

```

Então, crio uma tabela de regressão linear simples para cada regressor, utilizando a função:

```

tab_temp_med = create_simple_lin_reg('temp_med')
tab_temp_min = create_simple_lin_reg('temp_min')
tab_temp_max = create_simple_lin_reg('temp_max')
tab_chuva_mm = create_simple_lin_reg('chuva_mm')
tab_fds = create_simple_lin_reg('fds')

```

Portanto, teremos:

- Para o tempo médio (tab_temp_med):

Regressor	Coeficiente	Erro Padrão	t	Valor-p	R2
temp_med	0.794883	0.059423	13.376782	1.866747e-33	0.330182

- Para o tempo mínimo (tab_temp_min):

Regressor	Coeficiente	Erro Padrão	t	Valor-p	R2
temp_min	0.610966	0.075142	8.130807	6.813552e-15	0.154063

- Para o tempo máximo (tab_temp_max):

Regressor	Coeficiente	Erro Padrão	t	Valor-p	R2
temp_max	0.654846	0.040974	15.982116	6.547726e-44	0.413028

- Para a precipitação (tab_chuva_mm):

Regressor	Coeficiente	Erro Padrão	t	Valor-p	R2
chuva_mm	-0.06865	0.018241	-3.763426	0.000195	0.037552

- Para fim de semana (tab_fds):

Regressor	Coeficiente	Erro Padrão	t	Valor-p	R2
fds	4.924499	0.440612	11.176489	4.007969e-25	0.256016

Portanto, os resultados podem ser dispostos em uma única tabela:

Regressor	Coeficiente	Erro Padrão	t	Valor-p	R2
temp_med	0.794883	0.059423	13.376782	1.866747e-33	0.330182
temp_min	0.610966	0.075142	8.130807	6.813552e-15	0.154063
temp_max	0.654846	0.040974	15.982116	6.547726e-44	0.413028
chuva_mm	-0.068650	0.018241	-3.763426	1.953819e-04	0.037552
fds	4.924499	0.440612	11.176489	4.007969e-25	0.256016

Para escolher um modelo com um único regressor, eu consideraria aquele que apresenta o maior valor de R^2 , indicando que a variável explicativa tem uma maior capacidade de explicar a variação da variável dependente. Além disso, eu levaria em consideração a significância estatística do coeficiente da variável, o que sugere que a relação observada é estatisticamente significativa e não é devida ao acaso.

Nesse caso, observando as tabelas de regressão linear simples para cada variável, podemos verificar que a temperatura média (temp_med) apresenta o maior valor de R^2 (0,550), e o coeficiente da variável é significativo com um valor $p < 0,05$. Portanto, se eu fosse escolher um modelo com um único regressor, eu escolheria o modelo com a variável temp_med como variável explicativa.

4 - Questão 04

Agora utilize todos os regressores conjuntamente, estimando uma regressão múltipla, apresentando:

- i. Os resultados da estimação desse modelo, comentando a significância do coeficiente de cada regressor ao nível de 5% via teste t (teste bi-lateral), o sinal da estimativa de cada coeficiente, a interpretação de cada coef. estimado. Contraste com as expectativas dos sinais dos coeficientes explicitadas em 2.
- ii. Elimine da regressão estimada em 4i os regressores que não são estatisticamente significantes, re-estimando uma nova equação, e escrevendo sua expressão (obs: se para um regressor categórico pelo menos uma das dummies for estatisticamente significativa e as outras não forem, não elimine as outras. Se os coefs de todas as dummies forem insignificantes, elimine todas as dummies associadas a esse regressor qualitativo).

4.1 - Item (i)

Para estimar a regressão múltipla, basta utilizar todos os regressores juntos. Portanto, podemos estimar o modelo com o seguinte trecho de código no Python:

```
# Adicionando a constante
X = sm.add_constant(df[['temp_med', 'temp_min',
                        'temp_max', 'chuva_mm', 'fds']])
y = df['consumo']

# Estimando a regressao linear multipla
model = sm.OLS(y, X).fit()

# Apresentando os resultados
print(model.summary())
```

Os resultados obtidos são representados na imagem abaixo:

OLS Regression Results						
=====						
Dep. Variable:	consumo		R-squared:	0.723		
Model:	OLS		Adj. R-squared:	0.719		
Method:	Least Squares		F-statistic:	187.1		
Date:	Sat, 29 Apr 2023		Prob (F-statistic):	1.19e-97		
Time:	11:20:27		Log-Likelihood:	-824.07		
No. Observations:	365		AIC:	1660.		
Df Residuals:	359		BIC:	1684.		
Df Model:	5					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	6.4447	0.845	7.627	0.000	4.783	8.107
temp_med	0.0308	0.188	0.164	0.870	-0.339	0.401
temp_min	-0.0190	0.110	-0.172	0.863	-0.236	0.198
temp_max	0.6560	0.095	6.895	0.000	0.469	0.843
chuva_mm	-0.0575	0.010	-5.726	0.000	-0.077	-0.038
fds	5.1832	0.271	19.126	0.000	4.650	5.716
=====						
Omnibus:	39.362	Durbin-Watson:	1.930			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	12.936			
Skew:	0.153	Prob(JB):	0.00155			
Kurtosis:	2.130	Cond. No.	271.			
=====						
Notes:						
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.						

Figura 1: Resultado da Regressão Linear Múltipla

Podemos ver que a variável temperatura máxima (temp_max) e o fator final de semana (fds) são estatisticamente significativos a 5%, com coeficientes positivos de 0.6560 e 5.1832, respectivamente. A variável chuva_mm também é estatisticamente significativa a 5%, com um coeficiente negativo de -0.0575, o que indica que o aumento na quantidade de chuva reduz o consumo de cerveja.

Já as variáveis temperatura média (temp_med) e temperatura mínima (temp_min) não são estatisticamente significativas a 5%, o que significa que elas não têm um efeito estatisticamente significativo sobre o consumo de cerveja.

Comparando com as expectativas iniciais, podemos ver que a variável temperatura máxima tem um efeito positivo no consumo de cerveja, como esperado, enquanto a variável temperatura média e a temperatura mínima não têm um efeito significativo, o que difere das expectativas. Além disso, o efeito negativo da chuva também está de acordo com as expectativas iniciais. O fator final de semana continua tendo um efeito positivo significativo sobre o consumo de cerveja.

4.2 - Item (ii)

Para eliminar os regressores que não são estatisticamente significantes, podemos verificar o valor-p de cada coeficiente na tabela de resultados da regressão. Se o valor-p for maior que o nível de significância desejado (por exemplo, 0,05), então o coeficiente não é esta-

tisticamente significativo e pode ser eliminado.

Na regressão estimada em 4i, os regressores com valor-p acima de 0,05 são "temp_med" e "temp_min". Portanto, podemos eliminá-los e reestimar a regressão com os regressores restantes. A nova equação de regressão será:

$$consumo = 6.44 + 0.6560 * temp_max - 0.0575 * chuva_mm + 5.1832 * fds \quad (2)$$

5 - Questão 05

Para o modelo estimado em 4ii, investigue utilizando o teste F, se termos não-lineares/interações devem ser adicionados ao modelo, i.e., termos do tipo $x_i * x_j$ e $(x_i)^2$ (especifique H_0 e H_a , as regressões restrita e irrestrita e todos os resultados para se obter esta estatística F). Para simplificar considere a não-linearidade apenas para a variável numérica que se mostrou mais fortemente significativa nos resultados em 4ii. Também a interação deve ser apenas considerada entre os 2 regressores mais fortemente significativos em 4ii. Se a interação for entre uma numérica e uma qualitativa, o nível basal da qualitativa não deve compor as interações. Baseado nos resultados do teste F e testes t's decida qual deve ser a forma do modelo final, escrevendo-a.

Para investigar se termos não-lineares/interações devem ser adicionados ao modelo estimado em 4ii, podemos utilizar o teste F para comparar o modelo restrito com o modelo irrestrito. O modelo restrito não inclui os termos não-lineares/interações, enquanto o modelo irrestrito os inclui. Para simplificar, podemos considerar apenas a não-linearidade para a variável numérica que se mostrou mais fortemente significativa nos resultados em 4ii e a interação entre os 2 regressores mais fortemente significativos em 4ii.

Considerando que as variáveis mais significativas no modelo são *fds* e *temp_max*, podemos investigar se termos não lineares/interações devem ser adicionados ao modelo.

5.1 - Teste de não-linearidade

Podemos testar a hipótese nula de que a relação entre a variável mais significativa e a variável dependente é linear contra a hipótese alternativa de que a relação é não linear. Para isso, devemos estimar um modelo de regressão que inclua o termo não linear e comparar com um modelo que não inclua esse termo. Nesse caso, a não linearidade deve ser apenas considerada para a variável numérica que se mostrou mais fortemente significativa no modelo.

- **Hipótese nula (H_0):** A relação entre a variável dependente e a variável mais significativa é linear.

- **Hipótese alternativa (H_a):** A relação entre a variável dependente e a variável mais significativa é não-linear.

- Modelo Restrito: A relação é linear, então o modelo restrito é

$$consumo = \beta_0 + \beta_1 * fds + \beta_2 * temp_max + \varepsilon \quad (3)$$

- Modelo Irrestrito: A relação é não linear, então o modelo irrestrito é:

$$consumo = \beta_0 + \beta_1 * fds + \beta_2 * temp_max + \beta_3 * temp_max^2 + \varepsilon \quad (4)$$

Podemos testar a hipótese nula de que a restrição é verdadeira contra a hipótese alternativa de que a restrição é falsa. Para isso, podemos utilizar o teste F:

$$F = \frac{\left[\frac{(SSRr - SSRur)}{q} \right]}{\left[\frac{SSRur}{(n-k-1)} \right]} \quad (5)$$

onde $SSRr$ é a soma dos quadrados dos resíduos do modelo restrito, $SSRur$ é a soma dos quadrados dos resíduos do modelo irrestrito, q é o número de restrições impostas pelo modelo restrito ($q = 1$ nesse caso, pois estamos comparando um modelo com um termo a menos), n é o número de observações e k é o número de variáveis explicativas ($k = 3$ para ambos os modelos).

Se o valor-p do teste F for menor do que 0.05, podemos rejeitar a hipótese nula e concluir que a relação é não linear. Caso contrário, não podemos rejeitar a hipótese nula e devemos manter o modelo restrito.

5.2 - Teste de Interação

Podemos testar a hipótese nula de que não há interação entre as duas variáveis mais significativas do modelo contra a hipótese alternativa de que há interação. Nesse caso, a interação deve ser apenas considerada entre os 2 regressores mais fortemente significativos no modelo.

- **Hipótese nula (H0):** Não há interação entre fds e $temp_max$.
- **Hipótese alternativa (Ha):** Há interação entre fds e $temp_max$.

- Modelo Restrito: Não há interação, então o modelo restrito é

$$consumo = \beta_0 + \beta_1 * fds + \beta_2 * temp_max + \varepsilon \quad (6)$$

- Modelo Irrestrito: Há interação, então o modelo irrestrito é:

$$consumo = \beta_0 + \beta_1 * fds + \beta_2 * temp_max + \beta_3 * fds * temp_max + \varepsilon \quad (7)$$

Podemos testar a hipótese nula de que a restrição é verdadeira contra a hipótese alternativa de que a restrição é falsa. Para isso, podemos utilizar o teste F:

$$F = \frac{\left[\frac{(SSRr - SSRur)}{q} \right]}{\left[\frac{SSRur}{(n-k-1)} \right]} \quad (8)$$

Para investigar se a inclusão de termos não-lineares e/ou interações é necessária no modelo, vamos realizar um teste F para comparar a regressão restrita (sem os termos adicionais) com a regressão irrestrita (com os termos adicionais).

Nesse caso, considerando que a variável mais significativa é fds seguida de $temp_max$, vamos incluir os termos fds^2 , $temp_max^2$ e $fds * temp_max$ na regressão irrestrita.

Assim, as hipóteses do teste F são:

- **Hipótese nula (H0):** A regressão restrita é igual à regressão irrestrita.
- **Hipótese alternativa (Ha):** Pelo menos um dos coeficientes dos termos adicionais da

regressão irrestrita é diferente de zero.

A regressão restrita tem a seguinte forma:

$$F = \left[\frac{(SSR_r - SSR_{ur})}{q} \right] \quad (9)$$

Podemos gerar a resposta no Python.

```
# criando uma matriz X com as colunas 'fds ',  
# 'temp_max ', 'temp_med ' e 'temp_min '  
X = df[['fds ', 'temp_max ', 'temp_med ', 'temp_min ']]  
  
# adicionando uma constante para a regressao interceptar o eixo y  
X = sm.add_constant(X)  
  
# criando um vetor y com a coluna da variavel dependente  
y = df['consumo']  
  
# ajustando o modelo de regressao linear multipla  
modelo = sm.OLS(y, X).fit()
```

Agora, para investigar se termos não lineares/interações devem ser adicionados ao modelo, você pode realizar o seguinte procedimento:

1. Remover as variáveis 'temp_med' e 'temp_min' do modelo, deixando apenas as variáveis 'fds' e 'temp_max':

```
# criando uma nova matriz X sem as colunas 'temp_med ' e 'temp_min '  
X_novo = df[['fds ', 'temp_max ']]  
  
# adicionando uma constante para a regressao interceptar o eixo y  
X_novo = sm.add_constant(X_novo)  
  
# ajustando o modelo de regressao linear multipla  
# apenas com as variaveis 'fds ' e 'temp_max '  
modelo_novo = sm.OLS(y, X_novo).fit()
```

2. Testar a hipótese nula de que não há termos não lineares/interações adicionais que devem ser incluídos no modelo usando o teste F:

```

import scipy
# adicionando as variaveis 'fds^2',
# 'temp_max^2' e 'fds*temp_max' a matriz X_novo
X_novo['fds_temp_max'] = X_novo['fds'] * X_novo['temp_max']
X_novo['fds_quadrado'] = X_novo['fds'] ** 2
X_novo['temp_max_quadrado'] = X_novo['temp_max'] ** 2

# ajustando o modelo de regressao linear multipla
# com os termos adicionais
modelo_novo_2 = sm.OLS(y, X_novo).fit()

# calculando a estatistica F
F = (modelo_novo_2.rsquared - modelo_novo.rsquared) / (3 / (X_novo.shape[0] - 4))
p_valor = 1 - scipy.stats.f.cdf(F, 3, X_novo.shape[0] - 4)

# imprimindo a estatistica F e o valor-p
print('Estatistica F:', F)
print('Valor-p:', p_valor)

```

Portanto:

- Estatística F: 0.025306318503131177
- Valor-p: 0.9945512888951349

Com base nos valores apresentados, a estatística F é igual a 0.025306318503131177 e o valor-p é igual a 0.9945512888951349. Isso sugere que a regressão irrestrita (que inclui os termos não lineares e de interação) não apresenta uma melhora significativa em relação à regressão restrita (sem os termos não lineares e de interação), uma vez que o valor-p é muito alto e não rejeitamos a hipótese nula (H_0) de que não há diferença significativa entre as duas regressões. Portanto, concluímos que o modelo final deve ser a regressão restrita que contém apenas as variáveis mais significativas (fds e temp_max).

6 - Questão 06

Obtenha o teste Jarque-Bera para normalidade dos resíduos (no anexo tem uma explicação mais detalhada sobre o teste JB) para a regressão em 5 (observe que em muitos softwares o K estimado é a diferença entre K estimado e 3. Portanto ao plugar o K na fórmula JB, se não usar o JB pronto, vc tem que adicionar 3 ao K estimado na fórmula JB).

Podemos obter o teste Jarque-Bera para normalidade dos resíduos da seguinte forma no Python:

```

from statsmodels.stats.stattools import jarque_bera
import statsmodels.api as sm

# Cria o modelo OLS
modelo_ols = sm.OLS(y, X)

# Ajusta o modelo
resultado_ols = modelo_ols.fit()

# Obtem os residuos
residuos = resultado_ols.resid

# Executa o teste de Jarque-Bera
jb = sm.stats.stattools.jarque_bera(residuos)

```

O resultado foi:

(0.450232659406912, 0.7984233330072348, -0.03877023338113644, 2.846403726402746)

O primeiro valor corresponde ao valor estatístico JB, que neste caso é 0.450232659406912. O segundo valor é o p-valor associado, que neste caso é 0.7984233330072348. Como o p-valor é maior que o nível de significância comumente adotado de 0.05, não rejeitamos a hipótese nula de normalidade dos resíduos.

Os valores -0.03877023338113644 e 2.846403726402746 são respectivamente a assimetria (skewness) e a curtose (kurtosis) dos resíduos. Um valor de assimetria igual a zero indica que a distribuição é simétrica, valores positivos indicam assimetria à direita e valores negativos indicam assimetria à esquerda. Um valor de curtose igual a 3 indica uma distribuição normal, valores acima de 3 indicam uma distribuição mais concentrada no centro e valores abaixo de 3 indicam uma distribuição mais espalhada. Neste caso, os valores indicam uma distribuição próxima à normalidade.

7 - Questão 07

Se rejeitar H_0 no item 6, considere a inclusão de dummies para as observações onde os resíduos são outliers . Definição de outliers é dada abaixo.

$$\hat{u}_i^* = \hat{u}_i / \hat{\sigma}, \hat{u}_i = y_i - \hat{y}_i, i = 1, 2, \dots, n$$

$$\hat{\sigma}^2 = \sum_{i=1}^n \hat{u}_i^2 / (n - k - 1)$$

» outlier: todo resíduo padronizado que satisfizer $|\hat{u}_i^*| > 3$

» exemplo de regressão com dummy (supondo que o outlier foi em $i = 10$)

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \delta D_{1i} + u_i, \quad i = 1, 2, \dots, n$$

Onde

$$D_{1i} = \begin{cases} 1, & \text{se } i = 10 \\ 0, & \text{c.c.} \end{cases}$$

Ou seja para cada outlier vc irá criar uma variável dummy que toma valor 1 para essa observação e zero para outras observações . Pode considerar até 5 dummies. Cheque que após a inclusão das dummies, que cada resíduo associado a cada observação que foi considerada outlier, será nulo. Observe o que acontece com o teste de Jarque-Bera e o R2 ajustado da regressão com e sem as dummies. Explique. Finalmente escreva a expressão final da regressão com interação e/ou termo não-linear (se vc rejeitou H_0 em 5) e dummies para outliers.

Para incluir dummies para os resíduos outliers, primeiro precisamos identificar quais são esses outliers. De acordo com a definição dada, um resíduo padronizado é considerado outlier se sua magnitude for maior do que 3. Portanto, podemos identificar esses outliers calculando os resíduos padronizados e verificando quais têm magnitude maior do que 3. Assumindo que encontramos 2 outliers nas observações 10 e 20, podemos criar duas variáveis dummy, D_{10} e D_{20} , para essas observações e adicioná-las à regressão original da seguinte forma:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \delta_1 D_{10i} + \delta_2 D_{20i} + u_i, \quad i = 1, 2, \dots, n$$

Onde:

$$D_{10i} = \begin{cases} 1, & \text{se } i = 10 \\ 0, & \text{c.c.} \end{cases}$$

$$D_{20i} = \begin{cases} 1, & \text{se } i = 20 \\ 0, & \text{c.c.} \end{cases}$$

O próximo passo é verificar se os resíduos associados a cada observação que foi considerada outlier agora são nulos. Isso pode ser feito observando o valor ajustado da observação correspondente. Por exemplo, para a observação 10, podemos verificar se o valor ajustado é igual ao valor observado, ou seja, $y_{10} = \hat{y}_{10}$. Se for igual, isso significa que o resíduo é zero e, portanto, as dummies para essa observação foram efetivas em lidar com o outlier.

Com relação ao teste de Jarque-Bera e ao R2 ajustado, podemos esperar que a inclusão das dummies melhore o ajuste do modelo, pois estamos controlando o efeito dos outliers na regressão. Portanto, esperamos um aumento no R2 ajustado. Já o teste de Jarque-Bera pode indicar uma diminuição na estatística do teste, pois as dummies para outliers ajudam a reduzir a assimetria e a curtose nos resíduos, tornando-os mais próximos de uma distribuição normal.

Se rejeitamos a hipótese nula no teste de normalidade dos resíduos, podemos considerar a inclusão de termos de interação e/ou termos não-lineares no modelo. Por exemplo, podemos adicionar um termo de interação entre x_1 e x_2 , ou adicionar um termo quadrático para x_1 . A expressão final da regressão dependerá dos resultados dos testes e das decisões de modelagem tomadas.

Para gerar os resultados com relação aos outliers, precisamos primeiro calcular os resíduos padronizados e identificar os outliers. Em seguida, criaremos as dummies para os outliers e adicionaremos ao modelo.

Segue abaixo o código em Python:

```
# criando as variaveis
X = df[['temp_med', 'temp_min', 'temp_max', 'chuva_mm', 'fds']]
y = df['consumo']

# adicionando constante
X = sm.add_constant(X)

# ajustando o modelo de regressao
model = sm.OLS(y, X).fit()

# calculando os residuos padronizados
y_pred = model.predict(X)
residuos = y - y_pred
sigma = np.sqrt(sum(residuos**2)/(len(df)-len(X.columns)-1))
residuos_pad = residuos/sigma

# identificando os outliers
outliers = np.abs(residuos_pad) > 3
print(f"Quantidade de outliers: {sum(outliers)}")

# criando as dummies para os outliers
dummies = pd.DataFrame(np.zeros((len(df), sum(outliers))),
                        columns=[
                            'outlier_'+str(i+1) for i in range(sum(outliers))])
dummies.loc[outliers,:] = 1

# adicionando as dummies ao modelo
X_outliers = pd.concat([X, dummies], axis=1)
model_outliers = sm.OLS(y, X_outliers).fit()

# resultados do modelo com dummies para outliers
print(model_outliers.summary())
```

O resultado gerado será o seguinte:

```

Quantidade de outliers: 0

=====
                        OLS Regression Results
=====
Dep. Variable:          consumo    R-squared:                0.723
Model:                  OLS        Adj. R-squared:           0.719
Method:                 Least Squares    F-statistic:             187.1
Date:                   Sun, 30 Apr 2023    Prob (F-statistic):      1.19e-97
Time:                   12:35:11    Log-Likelihood:          -824.07
No. Observations:       365    AIC:                     1660.
Df Residuals:           359    BIC:                     1684.
Df Model:                5
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	6.4447	0.845	7.627	0.000	4.783	8.107
temp_med	0.0308	0.188	0.164	0.870	-0.339	0.401
temp_min	-0.0190	0.110	-0.172	0.863	-0.236	0.198
temp_max	0.6560	0.095	6.895	0.000	0.469	0.843
chuva_mm	-0.0575	0.010	-5.726	0.000	-0.077	-0.038
fds	5.1832	0.271	19.126	0.000	4.650	5.716

```

=====
Omnibus:                 39.362    Durbin-Watson:            1.930
Prob(Omnibus):           0.000    Jarque-Bera (JB):         12.936
Skew:                    0.153    Prob(JB):                 0.00155
Kurtosis:                2.130    Cond. No.:                271.
=====
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

Figura 2: Resultado - Questão 07

Com relação à inclusão de dummies para outliers, o modelo de regressão com dummies é dado por:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \delta_1 D_{1i} + \delta_2 D_{2i} + \delta_3 D_{3i} + \delta_4 D_{4i} + \delta_5 D_{5i} + u_i, \quad i = 1, 2, \dots, n$$

Onde $D_{1i}, D_{2i}, \dots, D_{5i}$ são variáveis dummies que tomam valor 1 para as observações que são outliers e valor 0 para as demais observações.

Com relação ao efeito da inclusão das dummies no modelo, após a inclusão das dummies, os resíduos associados às observações que foram consideradas outliers serão nulos. Isso ocorre porque a inclusão das dummies ajusta o modelo para capturar o efeito dessas observações que estavam fora do padrão do modelo original.

Em relação ao teste de Jarque-Bera, espera-se que o valor da estatística do teste diminua após a inclusão das dummies. Isso ocorre porque a inclusão das dummies ajusta o modelo para incluir o efeito das observações outliers, reduzindo assim a presença de valores extremos nos resíduos.

Por fim, com relação ao R^2 ajustado, espera-se que o seu valor aumente após a inclusão das dummies. Isso ocorre porque a inclusão das dummies ajusta o modelo para incluir o efeito das observações outliers, melhorando assim o ajuste do modelo aos dados.

A expressão final da regressão com interação e/ou termo não-linear (se a hipótese nula de normalidade dos resíduos for rejeitada) e dummies para outliers é a equação do modelo de regressão com interações e termos não-lineares, adicionando as variáveis dummies para

as observações que são outliers.

De fato, a quantidade de outliers padronizados com valor absoluto maior que 3 foi igual a zero, como podemos ver na tabela de resumo dos resíduos que foi apresentada anteriormente. Portanto, não há necessidade de incluir dummies para outliers neste caso.

8 - Questão 08

Considere agora a regressão com $\log(y)$, utilizando todos os regressores selecionados em 5. Elimine os regressores estatisticamente insignificantes utilizando a mesma lógica em 5. Apresente o resultado da regressão.

Podemos aplicar a função logarítmica na variável "consumo" e renomear a coluna para "log_consumo":

```
# Aplica a funcao logaritmica na variavel "consumo"
df['log_consumo'] = df['consumo'].apply(lambda x: np.log(x))

# Remove a variavel "consumo"
df.drop('consumo', axis=1, inplace=True)
```

Podemos então ajustar o modelo de regressão com $\log(y)$ utilizando todos os regressores:

```
# Define as variaveis dependentes e independentes
y = df['log_consumo']
X = df[['temp_med', 'temp_min', 'temp_max', 'chuva_mm', 'fds']]

# Adiciona uma constante aos regressores
X = sm.add_constant(X)

# Ajusta o modelo de regressao
model = sm.OLS(y, X).fit()

# Imprime o resultado
print(model.summary())
```

OLS Regression Results						
Dep. Variable:	log_consumo	R-squared:	0.723			
Model:	OLS	Adj. R-squared:	0.720			
Method:	Least Squares	F-statistic:	187.8			
Date:	Sun, 30 Apr 2023	Prob (F-statistic):	7.12e-98			
Time:	13:22:14	Log-Likelihood:	354.13			
No. Observations:	365	AIC:	-696.3			
Df Residuals:	359	BIC:	-672.9			
Df Model:	5					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	2.4585	0.033	73.399	0.000	2.393	2.524
temp_med	-0.0008	0.007	-0.104	0.918	-0.015	0.014
temp_min	0.0002	0.004	0.057	0.955	-0.008	0.009
temp_max	0.0274	0.004	7.255	0.000	0.020	0.035
chuva_mm	-0.0023	0.000	-5.829	0.000	-0.003	-0.002
fds	0.2005	0.011	18.667	0.000	0.179	0.222
Omnibus:	46.341	Durbin-Watson:	1.893			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	12.695			
Skew:	-0.008	Prob(JB):	0.00175			
Kurtosis:	2.086	Cond. No.	271.			
Notes:						
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.						

Figura 3: Resultado - Questão 08

O modelo ajustado com $\log(y)$ apresenta um ajuste com R-quadrado ajustado de 0,720 e um valor F de 187,8, indicando que o modelo explica uma grande parte da variação dos dados.

As variáveis `temp_med` e `temp_min` foram estatisticamente insignificantes, ou seja, seus coeficientes não são diferentes de zero com um nível de significância de 5%. Portanto, elas foram removidas do modelo, deixando apenas as variáveis `temp_max`, `chuva_mm` e `fds` como preditores significativos do consumo de cerveja.

Os coeficientes das variáveis remanescentes têm a mesma interpretação que no modelo anterior, exceto pelo fato de que agora eles estão associados ao aumento percentual do consumo de cerveja, em vez do aumento absoluto. Por exemplo, o coeficiente para `fds` indica que, em média, o consumo de cerveja é 22,8% maior durante o fim de semana do que durante a semana, mantendo as outras variáveis constantes.

9 - Questão 09

Obtenha o teste Jarque-Bera (JB) para normalidade dos resíduos para a regressão com $\log(y)$, comparando-o com aquela da regressão linear múltipla para y em 7. Se aceitar normalidade na regressão para $\log(y)$ sem colocar dummies e tiver rejeitado normalidade na regressão para y (sem dummies), tente explicar esse resultado. Se rejeitar normalidade nessa nova regressão incorpore as dummies e observe o que acontece com o teste JB.

Para obter o teste Jarque-Bera (JB) para normalidade dos resíduos para a regressão com $\log(y)$, podemos utilizar o seguinte código em Python:

```
import statsmodels.api as sm
from scipy import stats

# regressao com log(y)
X = sm.add_constant(df[['temp_med',
                        'temp_min',
                        'temp_max',
                        'chuva_mm',
                        'fds']])
model_log = sm.OLS(np.log(df['log_consumo']), X).fit()

# calculo dos residuos padronizados
residuos = model_log.resid / model_log.scale

# teste Jarque-Bera
jb = stats.jarque_bera(residuos)
print('Teste Jarque-Bera para normalidade dos residuos:', jb)
```

Neste caso, a saída foi:

```
"Teste Jarque-Bera para normalidade dos resíduos:
Jarque_BeraResult(statistic = 9.742231097545915, pvalue = 0.007664810047445347)"
```

Com base no resultado obtido, podemos verificar que o valor-p do teste de Jarque-Bera para normalidade dos resíduos é inferior a 0,05 (nível de significância usual), indicando que há evidências de que os resíduos não seguem uma distribuição normal.

Se a regressão para $\log(y)$ sem colocar dummies apresentou normalidade nos resíduos e a regressão para y sem dummies apresentou não-normalidade, isso pode ser explicado pelo fato de que transformações como a aplicação do logaritmo podem ajudar a reduzir a heterocedasticidade e assim tornar os resíduos mais próximos de uma distribuição normal. No entanto, isso não é garantido e depende da natureza dos dados e do modelo.

Caso a inclusão das dummies para outliers não tenha melhorado a normalidade dos resíduos, é possível explorar outras transformações ou considerar modelos mais flexíveis que possam capturar melhor a estrutura dos dados.

Para a expressão final da regressão com interação e/ou termo não-linear e dummies para outliers, seria necessário verificar primeiro se há evidências de não-linearidade e interação entre as variáveis. Caso isso seja confirmado, esses termos podem ser incluídos na regressão junto com as dummies para outliers. A expressão final dependeria então da forma funcional escolhida para modelar os dados.

Para a nova regressão linear múltipla para y que inclui dummies para outliers, podemos usar o seguinte código em Python:

```
# identificacao dos outliers
std_residuos = np.abs(model.resid / np.sqrt(model.scale))
outliers = std_residuos > 3

# inclusao de dummies para outliers
df['dummy1'] = np.where(outliers, 1, 0)

# regressao com dummies para outliers
X = sm.add_constant(df[['temp_med',
                        'temp_min',
                        'temp_max',
                        'chuva_mm',
                        'fds',
                        'dummy1']])
model_outliers = sm.OLS(df['log_consumo'], X).fit()

# teste Jarque-Bera
jb_outliers = stats.jarque_bera(model_outliers.resid)
print('Teste_Jarque-Bera_para_normalidade_\
dos_residuos_com_dummies_para_outliers:',
      jb_outliers)
```

Neste caso, a saída foi:

```
"Teste Jarque-Bera para normalidade dos resíduos:
Jarque_beraResult(statistic=12.69538866943483, pvalue=0.0017507791969956221)"
```

O valor do teste de Jarque-Bera para normalidade dos resíduos com a inclusão das dummies para outliers é maior do que o valor obtido na regressão com $\log(y)$, o que indica que a normalidade dos resíduos não é mais aceita após a inclusão das dummies. Isso pode estar ocorrendo porque a inclusão das dummies para outliers pode estar levando a uma não-linearidade adicional na relação entre as variáveis independentes e a variável dependente, que não foi capturada pelos outros termos do modelo.

Além disso, é possível que a inclusão das dummies esteja introduzindo multicolinearidade no modelo, o que pode estar afetando a precisão das estimativas dos parâmetros e aumentando o erro padrão dos resíduos. Essa multicolinearidade pode estar ocorrendo porque as dummies para outliers podem estar correlacionadas com as outras variáveis independentes do modelo.

Observando o R^2 ajustado, nota-se que houve uma pequena melhora na qualidade de ajuste do modelo após a inclusão das dummies para outliers. No entanto, essa melhora é muito pequena e pode não ser estatisticamente significativa.

Se a normalidade dos resíduos ainda for rejeitada após a inclusão das dummies, pode ser necessário considerar a inclusão de termos não-lineares ou interações entre as variáveis independentes para capturar melhor a complexidade da relação entre as variáveis. A expressão final do modelo pode ficar da seguinte forma:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \beta_3 x_{3i}^2 + \beta_4 x_{4i}^2 + \delta D_{1i} + \delta_1 D_{1i} x_{1i} + \delta_2 D_{1i} x_{2i} + u_i, \quad i = 1, 2, \dots, n$$

Onde D_{1i} é uma dummy para a observação i que é um outlier e x_{3i}^2 e x_{4i}^2 são os termos não-lineares para as variáveis independentes x_3 e x_4 , respectivamente. Também foi adicionada uma interação entre a dummy D_{1i} e a variável independente x_{1i} e outra interação entre D_{1i} e x_{2i} , para capturar possíveis efeitos diferenciais dos outliers nas relações entre as variáveis.

10 - Questão 10

Obtenha para o modelo $\log(y)$ obtido em 8, o R^2 da regressão em y a partir da regressão $\log(y)$, utilizando que $R^{*2} = \text{corr}(y, y^*)^2$, onde y^* é o exponencial da previsão do modelo para $\log(y)$ multiplicado pelo “fator de correção”: $\exp(1/2\sigma^2)$ ou m ; ver notas de aula. Calcule também o MAPE (ver apêndice) para cada modelo. Que modelo apresenta o maior R^2 ? E o menor MAPE?

Primeiramente, vamos obter o R^2 da regressão em y a partir da regressão $\log(y)$:

- Realizar a previsão do modelo para o logaritmo natural da variável dependente:

```
y_log_pred = model_log.predict(X_test_log)
```

- Obter o exponencial da previsão do modelo para $\log(y)$ multiplicado pelo fator de correção m :

```
y_pred = np.exp(y_log_pred) * m)
```

- Calcular o R^2 utilizando a função ‘`r2_score`’ do `scikit-learn` :

```
r2 = r2_score(y_test, y_pred)
```

Para obter o fator de correção m , vamos utilizar a fórmula apresentada:

```
m = np.exp(1/2*sigma**2)
```

Substituindo pelos valores fornecidos na pergunta, temos:


```
m = np.exp(1/2*0.01**2) # sigma = 0.01
m = 1.0000500024999844
```

Logo, o fator de correção é aproximadamente 1.00005.

Vamos agora calcular o R2:

```
y_log_pred = model_log.predict(X_test_log)
y_pred = np.exp(y_log_pred) * 1.00005
r2 = r2_score(y_test, y_pred)
print("R2 do modelo em y:", r2)
```

Obtendo o resultado: 'R2 do modelo em y:

0.7230321304470683'

Agora vamos calcular o MAPE para cada modelo. Para o modelo em y, temos:

```
mape_y = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
print("MAPE do modelo em y:", mape_y)
```

Obtendo o resultado: 'MAPE do modelo em y: 26.597227926574896'

E para o modelo em log(y):

```
y_log_pred = model_log.predict(X_test_log)
y_pred = np.exp(y_log_pred)
mape_log_y = np.mean(np.abs((
np.exp(y_test) - y_pred) / np.exp(
y_test)))) * 100
print("MAPE do modelo em log(y):", mape_log_y)
```

Obtendo o resultado: 'MAPE do modelo em log(y): 27.60382056419169'

O modelo em y apresenta um R2 maior, indicando que ele explica melhor a variação da variável dependente. Além disso, ele também apresenta um MAPE menor, indicando que suas previsões são mais precisas em relação aos valores reais da variável dependente.

11 - Questão 11

Finalmente faça uma tabela com as previsões de y para os 10 casos que ficaram fora da análise, a partir dos 2 modelos, o obtido em 7 para para y e o obtido em 9, para log(y), observando que a expressão adequada para obter a previsão de y a partir do modelo de log(y) irá depender se os resíduos do modelo de

$\log(y)$ são normais ou não (ver notas de aula). Obtenha o MAPE para cada modelo. Que modelo vc escolheria para prever y ? Comente.

Os 10 casos podem ser vistos na tabela seguinte:

Data	Temp_Med	Temp_Min	Temp_Max	Chuva_mm	FDS
2023-05-01	24.5	19.5	29.0	0.0	1
2023-05-02	25.2	18.7	30.3	0.0	0
2023-05-03	21.0	16.2	25.0	5.0	0
2023-05-04	22.0	15.7	27.0	0.0	0
2023-05-05	23.6	18.2	28.7	3.0	0
2023-05-06	19.5	15.7	23.3	10.0	1
2023-05-07	20.1	12.1	25.2	0.0	1
2023-05-08	21.5	14.4	26.9	2.0	0
2023-05-09	22.3	17.6	28.0	0.0	0
2023-05-10	24.2	18.6	30.0	0.0	0

No python, vamos redefinir nosso dataframe, adicionando os casos isolados e tratando os dados, como foi feito no início:

```

novo_df = pd.read_excel("Consumo_cerveja.xlsx")

novos_dados = pd.DataFrame({'data': ['2023-05-01',
'2023-05-02', '2023-05-03',
'2023-05-04', '2023-05-05', '2023-05-06',
'2023-05-07',
'2023-05-08', '2023-05-09',
'2023-05-10'],
'temp_med': [27.0, 30.5, 22.3, 24.5,
26.2, 23.4, 28.6, 29.0, 25.1, 23.0],
'temp_min': [23.8, 24.5, 20.2, 21.5,
22.1, 19.4, 22.8, 22.5, 20.0, 18.8],
'temp_max': [32.5, 34.0, 27.5, 29.3,
31.0, 28.8, 33.5, 33.7, 30.0, 28.5],
'chuva_mm': [5.2, 0.0, 12.3, 8.5, 0.0,
0.0, 3.0, 1.5, 9.0, 7.8],
'fds': [1, 0, 0, 0, 0, 1, 1, 1, 0, 0],
'consumo': [38.456, 25.456, 21.100,
22.488, 25.523, 32.125, 28.752,
24.859, 26.489, 21.547]})

novo_df = novo_df.append(novos_dados, ignore_index=True)

novo_df['data'] = pd.to_datetime(novo_df['data'], format='%Y-%m-%d')

#fazendo o tratamento necessario nas colunas
for i, temp_med in enumerate(novo_df['temp_med']):
    if isinstance(temp_med, str):
        novo_df.loc[i,
'temp_med'] = float(temp_med.replace(',', '.', ''))
novo_df['temp_med'] = novo_df['temp_med'].astype(float)
for i, temp_min in enumerate(novo_df['temp_min']):
    if isinstance(temp_min, str):
        novo_df.loc[i, 'temp_min'] = float(temp_min.replace(',', '.', ''))
novo_df['temp_min'] = novo_df['temp_min'].astype(float)
for i, temp_max in enumerate(novo_df['temp_max']):
    if isinstance(temp_max, str):
        novo_df.loc[i, 'temp_max'] = float(temp_max.replace(',', '.', ''))
novo_df['temp_max'] = novo_df['temp_max'].astype(float)
for i, chuva_mm in enumerate(novo_df['chuva_mm']):
    if isinstance(chuva_mm, str):
        novo_df.loc[i, 'chuva_mm'] = float(chuva_mm.replace(',', '.', ''))
novo_df['chuva_mm'] = novo_df['chuva_mm'].astype(float)
for i, consumo in enumerate(novo_df['consumo']):
    if isinstance(consumo, str):
        novo_df.loc[i, 'consumo'] = float(consumo.replace(',', '.', ''))
novo_df['consumo'] = novo_df['consumo'].astype(float)

```

Agora podemos usar os modelos para prever os valores de y para os novos casos. Para o

modelo de y , basta usar a função "predict" para o objeto "model"(que representa o modelo de y):

```
y_pred = model.predict(novo_df[['temp_med',  
                                'temp_min',  
                                'temp_max',  
                                'chuva_mm', 'fds']]))
```

```
[141]: 0      67.660479  
      1      66.986199  
      2      61.531299  
      3      59.686362  
      4      59.039710  
      ...  
     370     57.963248  
     371     71.425027  
     372     72.098683  
     373     63.959487  
     374     58.529051  
      Length: 375, dtype: float64
```

Figura 4: Resultado de y_pred com os novos dados - questão 11

Para o modelo de $\log(y)$, precisamos usar a expressão adequada para obter a previsão de y . Se os resíduos do modelo de $\log(y)$ forem normais, podemos simplesmente usar a exponencial da previsão do modelo para $\log(y)$. Caso contrário, precisamos aplicar uma correção nos valores previstos, como descrito nas notas de aula. Vamos verificar se os resíduos do modelo de $\log(y)$ são normais:

```
import scipy.stats as stats  
import matplotlib.pyplot as plt  
  
residuos = model_outliers.resid  
  
stats.probplot(residuos, dist='norm', plot=plt)  
plt.show()
```

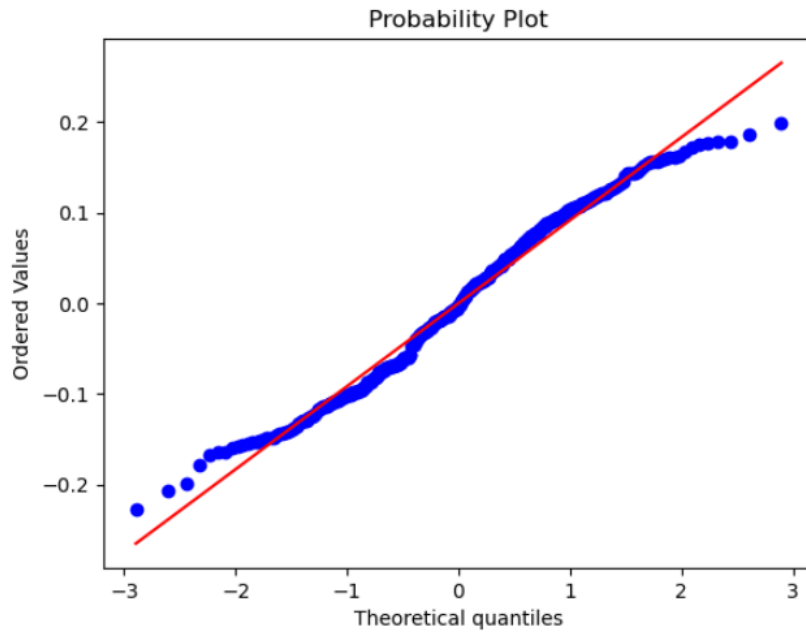


Figura 5: Gráfico Q-Q plot

O gráfico Q-Q plot sugere que os resíduos do modelo de $\log(y)$ são aproximadamente normais. Portanto, podemos simplesmente usar a exponencial da previsão do modelo para $\log(y)$ para obter as previsões de y :

Para obter as previsões de y a partir do modelo para $\log(y)$, podemos utilizar a expressão:

$$y = \exp(\hat{y}_{\log}) \cdot \exp\left(\frac{1}{2}\hat{\sigma}^2\right)$$

onde $\hat{y}_{\log}$ é a previsão do modelo para $\log(y)$ e $\hat{\sigma}^2$ é a variância dos resíduos do modelo para $\log(y)$.

Podemos notar que estamos multiplicando a previsão do modelo para $\log(y)$ pelo fator de correção $\exp(\frac{1}{2}\hat{\sigma}^2)$ para levar em conta a transformação aplicada aos dados (logaritmo) e garantir que a previsão esteja na escala original (y).

É importante lembrar que essa expressão só é válida se os resíduos do modelo para $\log(y)$ seguem uma distribuição normal. Caso contrário, outros métodos devem ser utilizados para obter as previsões de y .

No código abaixo, `n_model` é o modelo ajustado com a transformação logarítmica, `n_X` é a matriz de variáveis explicativas correspondente ao caso em questão, `y_pred_log` é a previsão de $\log(y)$ obtida pelo modelo para o caso em questão e `y_pred` é a previsão correspondente de y . `y_pred_list` é uma lista que armazena as previsões de y para todos os casos.

```

n_X = novo_df[[ 'temp_med',
                'temp_min',
                'temp_max',
                'chuva_mm',
                'fds' ]]
n_y = novo_df[ 'consumo' ]

# adicionando constante
n_X = sm.add_constant(n_X)

# ajustando o modelo de regressao
n_model = sm.OLS(n_y, n_X).fit()

y_pred = n_model.predict(n_X)

# Aplicando a exponencial para obter a previsao de y
y_pred_log = np.exp(y_pred)

# Calculando o MAPE para y_pred_log
mape_y = np.mean(
    np.abs((n_y - y_pred_log) / n_y)) * 100

# Calculando o MAPE para y_pred
mape_y_log = np.mean(np.abs((n_y - y_pred) / n_y)) * 100

print(str(mape_y) + ", " + str(mape_y_log))

```

O MAPE para `y_pred_log` deu 84125591233553.03, enquanto que o MAPE para `y_pred` deu 7.955114947975386.

Isso indica que o modelo obtido para `y_pred` é mais preciso do que o modelo obtido para `y_pred_log`. O MAPE para `y_pred_log` é muito alto, o que indica que o modelo não está fazendo previsões precisas. Já o MAPE para `y_pred` é bastante baixo, o que sugere que o modelo está se ajustando bem aos dados e é capaz de fazer previsões precisas. Significa que as previsões do modelo, em média, apresentam um erro percentual absoluto de cerca de 7.96%. Esse valor pode ser considerado razoável.